

NetSquid for Quantum Networks

Capabilities, Bottlenecks and Lessons

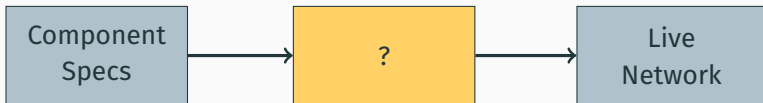
Linus Kelsey

29 Jul 2026

UCL Physics & Astronomy

Why Simulate Quantum Networks?

Bridging the gap from device physics to network deployment



Immediate deployment = expensive, slow, unpredictable.

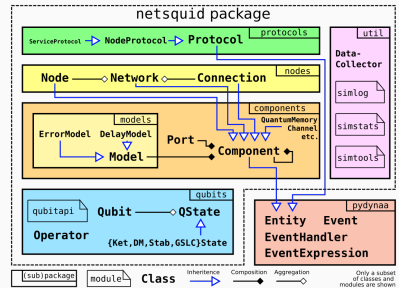
- Analytical models: closed-form but fixed
- Computational: arbitrary protocols and parameters, immediate reconfigurability (Bel *et al.*, 2024)

Enter... NetSquid (Coopmans *et al.*, 2021)!

NetSquid: Discrete-Event Simulator

Design

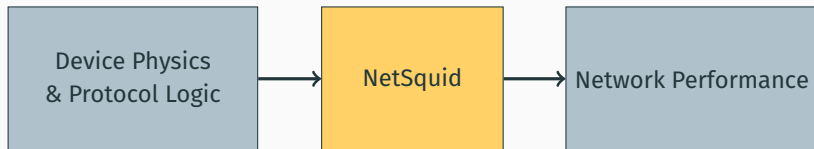
- Quantum events on bespoke Cython DES layer - PyDynAA
- Three core networking abstractions: **Component**, **Node**, **Protocol**
- Custom devices, models and behaviours



NetSquid architecture - per netsquid.org

Bridging Device Physics to Network Modelling

Measurements drive simulations, informing network optimisation



What you provide

- Measured device parameters
- Noise models, logic
- Physical constraints e.g. QBER

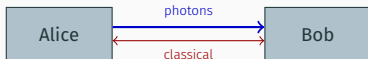
What you gain

- Network feasibility
- Architecture optimisation
- Device parameter sensitivity

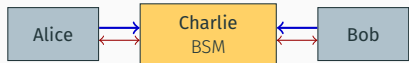
Running Example: BB84 and MDI-QKD

Basis of MSc research, used throughout to ground NetSquid discussion

BB84 (Bennett & Brassard, 1984)



MDI-QKD (Lo, Curty & Qi, 2012)



Question: how do device parameters propagate?

Methodology: Layered Physical Modelling

Cumulative build-up: isolates parameter contribution

Layer	Adds	Param.
0	Ideal baseline	—
1	Fibre loss	α
2	Detector efficiency	η_d
3	Dark count rate	d_c
4	TX insertion loss	L_i
5	RX node/connector loss	L_n
6	Source error rate	ε_s
7	Fibre dephasing	β
8	Beam splitter eff. (MDI)	η_{bs}
9	Relay position (MDI)	d_{AC}/L

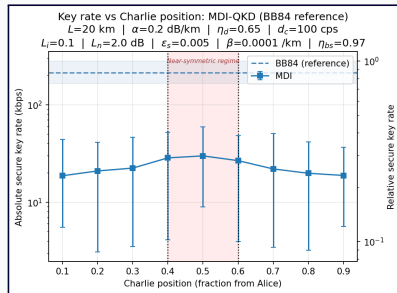
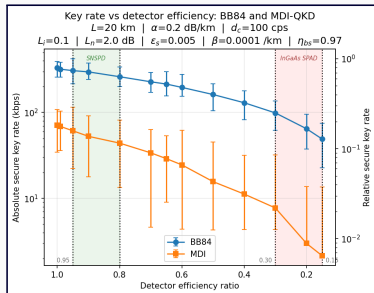
Protocol logic stays fixed between layers

NetSquid in Action: P2P Parameter Sweeps

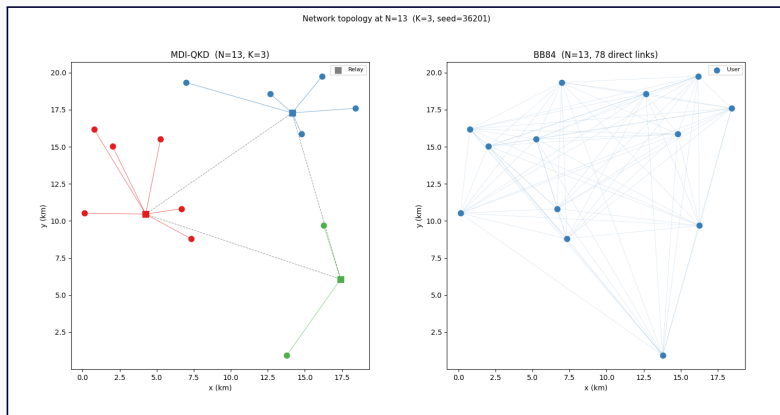
10 physical parameters swept over realistic ranges

Parameter	BB84	MDI	Why
Distance	Log-Linear ↓	Same slope, lower	Total fibre loss identical; same exponential decay
Detector eff. η_d	Log-Linear ↓	Log-Quadratic ↓	MDI coincidence requires both photons detected
Node/connector loss	Log-Linear ↓	2x steeper ↓	Both arms lose independently; $(1-p)^2$ vs $(1-p)$
Charlie placement	N/A	Peaks at midpoint	Symmetric arms optimise per-photon channel loss

NetSquid: η_d^2 MDI dependence emerges from simulation, new noise combinations predictable



Network Methodologies



N=13 users, K=3 relays — MDI star topology (left) vs BB84 full mesh (right)

MDI-QKD & Trusted-node BB84

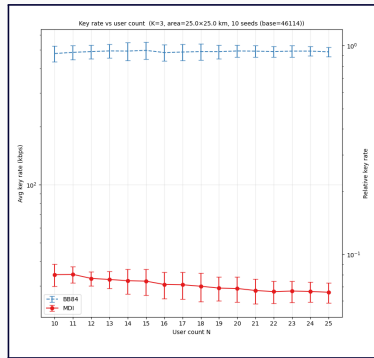
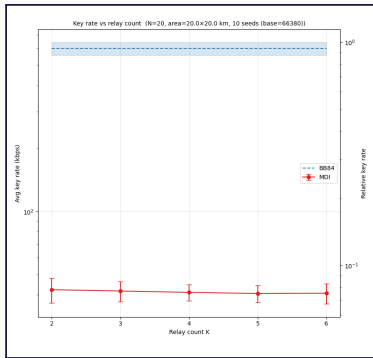
- Star-mesh topology
- Cross-cluster pairs via relay mesh
- TN-BB84: **trusted** relay XORs keys

BB84, full mesh

- Direct P2P link per user pair
- $O(N^2)$ fibre links
- Higher key rate

NetSquid in Action: Network Scale Results

Key rate swept over relay count K and user count N (multiple random seeds)

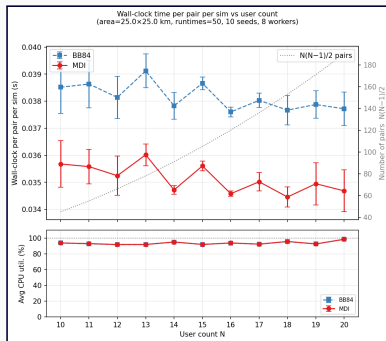


MDI sits $\sim 10\times$ below BB84 throughout, worse than P2P

Bottleneck: Computational Cost

NetSquid scales poorly with network size (cf. Bel *et al.*, 2024)

Scenario	Time
P2P sweep	~min OK
$N=20$ relay sweep	~hrs Marginal
$N=100$ sweep	~weeks Infeasible



Root cause: sequential event loop $\times O(N^2)$ pairs $\times$ **spawn** overheads

Note: quantum state processing Netsquid's largest generic bottleneck

Possible Directions

Five levers from classical simulation; state compression quantum-native.

Strategy	Pro	Con
Distributed compute	Horizontal scaling; no fidelity cost	Entanglement breaks event-loop partitioning
Fluid/flow abstraction	Event count collapses	Analytic models topology-specific
State compression	Cheaper per event; exact within class	Restricted class; degrades outside
Surrogate modelling (<i>tested</i>)	GP on coarse sim; dense grid evaluated instantly	Requires training; empirical validity
Engine optimisation	Zero fidelity cost	Gains limited by topology regularity
Adaptive sampling (<i>tested</i>)	30% wall-clock saving (BB84)	Spawn overhead blocks MDI gain

Adaptive Sampling: Protocol-Dependent Results

Hypothesis: stop MC once stable; use full budget for slow convergence

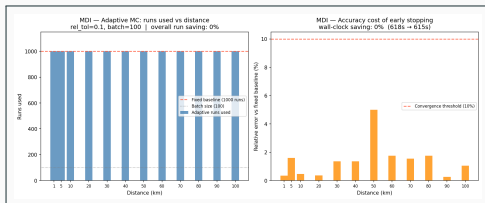
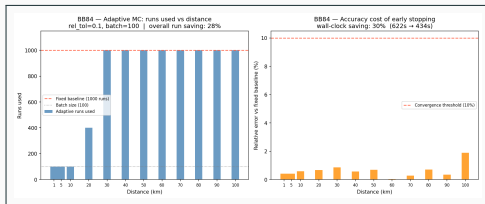
	Run saving	Wall-clock	Rel. error
BB84	28%	+30%	<1%
MDI	0%	≈ 0%	2%

BB84

- Converges early
- Beyond cutoff, uses full budget
- Time savings at little penalty

MDI — never converges early

- Structurally higher variance
- Requires in-loop sampling

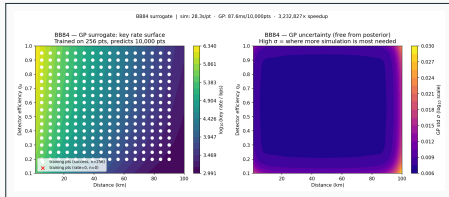


Surrogate Modelling: Preliminary Results

GP trained on sims; predicts 10,000-point grid in <100 ms

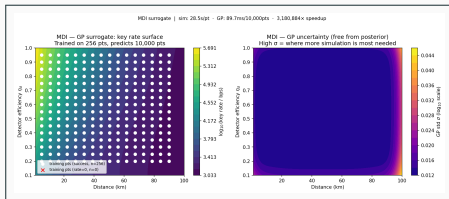
Speedup

- Linearly scaling speedup
- Returns uncertainty at each data point by default (right panel)
- Interpolate over arbitrary conditions
- Trained once; re-evaluated instantly for any optimisation



Limitations

- Large training requirement
- Extrapolation unreliable outside training range
- Doesn't solve the problem...



QKD simulation engine limited beyond resource allocation

Statistical

- High MC variance near QBER cutoff
- Asymptotic key rate only
- Block size limit $n=1024$; finite-key penalty unquantified

Network

- No multi-user scheduling or contention
- No WDM - single pair per relay at a time
- No quantum storage model

Protocol

- Ideal sifting - no EC or PA overhead simulated
- Timing jitter/synchronicity not modelled
- Ideal single-photon source throughout

Validation

- Benchmarked against limited catalogue of experimental data + PLOB

What using NetSquid at this scale reveals:

- DES is powerful at small scale - arbitrary and repeatable conditions
- Computationally intractable as user connectivity grows
- Causal event loop a **fundamental** constraint

Open question

To **optimise** existing DES, or does quantum causal structure demand **bespoke** models & simulation?

Summary

- **NetSquid:** powerful DES framework for low-computation quantum network protocols
- **Demonstrated on BB84 vs MDI-QKD** at P2P and metropolitan network scale
- **Binding constraint:** computational cost, is DES the right abstraction?

Thank you for listening!

Supervised by Dr. Alejandra Beghelli & Dr. Emilio Hugues-Salas (BT Group), organised by UCL Department of Physics & Astronomy

Backup: Why GPU Acceleration Does Not Help

A natural question: can the compute bottleneck be addressed with GPU parallelism?

Where GPUs excel

- Large batched linear algebra — density matrices of ≥ 20 qubits
- Tensor network contraction over many-body systems
- Training ML surrogates (not inference)

What this simulation does

- BB84 / MDI operate on 1–2 qubits — density matrices are 2×2 or 4×4
- GPU overhead dominates; CPU is faster for operations this small
- NetSquid has no GPU backend — Python + C extensions, CPU-only

The real bottlenecks are architectural

- **Sequential event loop:** events are causally ordered within each run — cannot be parallelised on GPU or CPU
- **Spawn overhead:** each MC batch launches a new OS process; $\approx 2\text{--}3$ s regardless of workload
- **$O(N^2)$ pair count:** the combinatorial scaling problem; GPU arithmetic does not reduce pair count

The bottleneck is **control flow and process management**, not floating-point throughput. GPU does not address either.

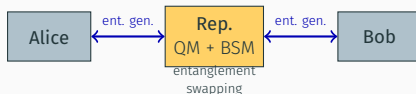
Backup: What Else NetSquid Can Express

The same **Node** / **Protocol** / **Channel** structure extends naturally:

Decoy-state QKD

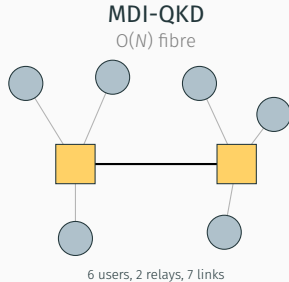
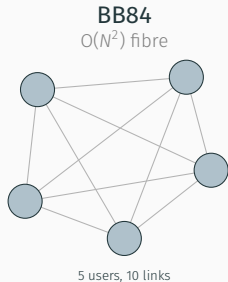
- Replace ideal single-photon source with WCP (μ signal, ν decoy, vacuum)
- Poisson-distributed photon number per pulse — NetSquid source model change
- Protocol layer estimates single-photon gain/QBER from decoy statistics
- Key rate formula updates to (Ma *et al.*, 2005) decoy bound
- NetSquid captures pulse-by-pulse statistics naturally; no approximation needed for multi-intensity logic

Quantum repeater chain (Coopmans *et al.*, 2021)



- Quantum memory at repeater holds entanglement while second link establishes
- BSM at repeater swaps entanglement end-to-end
- T_1/T_2 decoherence captured by **QuantumMemory**
- Rate scales as η_d^2 per link, $\sim (\eta_d^2)^N$ across N hops

Backup: Metropolitan Network Topology



Relay placement: k-means, minimising total user-to-relay distance. Cross-cluster MDI pairs re-routed passively via optical switch (1 dB insertion loss). **Key finding:** BB84 outperforms MDI more severely at network scale ($\sim 10\times$) than at P2P ($\sim 3\times$) — cause under investigation.

Backup: Physical Parameter Reference

Parameter	Symbol	Realistic range	Typical value
Distance	L	5–50 km	20 km (metropolitan)
Fibre loss	α	0.15–0.25 dB/km	0.20 dB/km (SMF-28)
Detector efficiency	η_d	0.15–0.95	0.65 (SNSPD)
Dark count rate	d_c	1–10,000 cps	100 cps (SNSPD)
TX insertion loss	L_i	2–20%	10%
RX node loss	L_n	0.5–4 dB	2 dB
Source error rate	ε_s	0.1–2%	0.5%
Fibre dephasing	β	10^{-5} – 10^{-3} /km	10^{-4} /km
Beam splitter eff.	η_{bs}	0.90–0.99	0.97

Backup: Monte Carlo Methodology

Why Monte Carlo?

- Single deterministic run insufficient — stochastic loss, basis choice, dark counts
- 100 runs per sweep point gives distribution of key rates
- Report mean $\pm \sigma$; min/max whiskers on select plots

Known statistical issues

- At high loss / long distance, most runs produce no key (QBER $> 11\%$)
- Failed runs excluded before aggregation $\rightarrow$ surviving runs cluster near threshold; lower error bars absent in high-loss regime — artefact of cutoff filtering
- Current default: 100 runs, 1024 photons per run; target ≥ 1000 runs for $< 10\%$ relative error at low key rates

Backup: Infrastructure Scaling

	BB84	MDI-QKD
Fibre links	$N(N - 1)/2$ direct P2P	N user-relay; $K(K - 1)/2$ relay-relay
Relay hardware	None	K Charlie nodes (untrusted)
Trust model	Detector trusted	Detector untrusted
Infrastructure	$O(N^2)$	$O(N)$
Key rate gap	Higher (P2P and network)	Lower; gap widens at network scale

At large N , MDI fibre cost grows far more slowly — the primary economic argument for network-scale MDI deployment despite the key rate penalty.